



PluriTAL

Traitement Statistiques des données 2025-2026 (M1)

Projet de classification : critiques de films

Enseignant référent : Damien Nouvel

Etudiantes : LE BAIL Emmy, ZENG Youjia, ZHENG Ting

SOMMAIRE

1. Introduction.....	3
2. Présentation du corpus.....	3
2.1 Origine des données.....	3
2.2 Composition et répartition.....	3
3. Prétraitement et Vectorisation (Feature Engineering).....	4
4. Expérimentations et Algorithmes.....	4
4.1. Naives Bayes.....	4
4.2. KNN (K-Nearest Neighbors).....	5
4.3. WEKA / J48.....	5
4.4. WEKA / Random forest.....	6
5. Analyse des Résultats et Comparaison.....	6
6. Conclusion.....	7

1. Introduction

L'analyse de sentiments (*Sentiment Analysis*) constitue aujourd'hui l'un des axes de recherche les plus actifs du Traitement Automatique des Langues (TAL). En permettant de transformer des données textuelles brutes en informations structurées sur l'opinion des utilisateurs, elle trouve des applications dans de nombreux domaines, allant de la veille commerciale à l'analyse des réseaux sociaux. Dans ce contexte, la classification automatique de textes selon leur polarité (positive ou négative), représente une tâche fondamentale, dont la résolution soulève néanmoins des défis importants liés à la richesse et à l'ambiguïté du langage naturel.

Ce travail s'inscrit dans cette problématique en se concentrant sur la classification de critiques cinématographiques issues du corpus IMDB en anglais. L'objectif est d'évaluer et de comparer l'efficacité de quatre approches algorithmiques classiques de l'apprentissage automatique supervisé : Naive Bayes, K-Nearest Neighbors (KNN), J48 (arbre de décision) et Random Forest. Nous cherchons ainsi à répondre à la question suivante : lequel de ces algorithmes offre les meilleures performances pour identifier l'opinion d'un spectateur à partir de son texte, au regard de métriques d'évaluation telles que la précision, le rappel et la F-mesure ?

Afin d'y répondre, nous présenterons dans un premier temps la constitution du corpus et les étapes de prétraitement appliquées aux données brutes. Nous exposerons ensuite la méthodologie de vectorisation retenue pour la représentation numérique des textes. Enfin, nous analyserons et comparerons les performances obtenues par chacun des classifieurs, afin d'en dégager les forces et les limites respectives dans le cadre de cette tâche de classification binaire.

2. Présentation du corpus

2.1 Origine des données

Pour ce projet, nous avons utilisé le dataset [IMDB Movie Reviews](#), récupéré sur Kaggle. Ce corpus est devenu une référence en TAL pour l'analyse de sentiments : il rassemble des critiques extraites d'Internet Movie Database et offre un volume suffisant pour entraîner et évaluer des modèles robustes.

2.2 Composition et répartition

Le corpus IMDB contient 50 000 critiques de films annotées manuellement, réparties de manière équilibrée entre commentaires positifs et négatifs (50 % / 50 %).

Pour nos expériences, nous avons divisé le corpus à l'aide du script [split_dataset.py](#) afin de disposer d'un jeu de validation dédié à l'ajustement des hyperparamètres. La nouvelle répartition est la suivante : 70 % pour l'entraînement, 15 % pour la validation et 15 % pour le test. Le jeu de validation sert à optimiser les paramètres des modèles, tandis que le jeu de test est strictement réservé à l'évaluation finale des performances.

3. Prétraitement et Vectorisation (Feature Engineering)

Avant l'apprentissage des modèles, plusieurs prétraitements sont appliqués aux textes. Ces opérations sont regroupées dans le script [pretraitement.py](#) contenant une fonction `pretraitement`, qui sera réutilisée dans les différentes étapes d'apprentissage des modèles. Les balises HTML sont supprimées avec BeautifulSoup4, les textes sont convertis en minuscules et les ponctuations sont retirées avec des expressions régulières. Les mots vides (*stopwords*) sont supprimés via NLTK, à l'exception des termes de négation (no, not, never) qui sont conservés, car ils influencent fortement la polarité des critiques.

Pour les expériences réalisées avec WEKA, les fichiers CSV sont ensuite convertis au format ARFF avec un script dédié ([csv2arff.py](#)). Cette conversion applique également le même prétraitement afin de garantir une cohérence entre les expériences Scikit et WEKA. Le script génère finalement des fichiers `.arff` contenant les critiques prétraitées et leurs labels de sentiment.

Pour la vectorisation des textes, nous utilisons `TfidfVectorizer` de scikit-learn pour les modèles développés en Python. Dans WEKA, la transformation des textes en vecteurs est réalisée avec le filtre `StringToWordVector`.

4. Expérimentations et Algorithmes

Dans cette partie, nous expérimentons plusieurs algorithmes de classification pour l'analyse de sentiments sur le dataset IMDB, en suivant la répartition décrite en section 2.2. Nous utilisons les implémentations de scikit-learn pour les modèles Naive Bayes et KNN, ainsi que les outils de WEKA pour les algorithmes J48(Décision Tree) et Random Forest. Tous les modèles s'appuient sur le même prétraitement des données afin de garantir une comparaison cohérente des résultats. Chaque modèle est entraîné sur le jeu d'entraînement, ajusté sur le jeu de validation, puis évalué définitivement sur le jeu de test.

4.1. Naives Bayes

L'implémentation de cet algorithme est détaillée dans le fichier [NaiveBayes.py](#).

Les données de train, validation et test sont chargées avec Pandas, puis les textes sont prétraités afin de nettoyer les phrases avant l'apprentissage. Les critiques sont ensuite transformées en vecteurs numériques avec `TfidfVectorizer` (TF-IDF).

Plusieurs paramètres sont testés, notamment différentes valeurs de `alpha` et plusieurs types de `n-grams`, afin de sélectionner la meilleure configuration grâce au jeu de validation. Les résultats montrent que la meilleure performance est obtenue avec `alpha = 0.1` et `ngram_range = (1, 2)`, avec une `accuracy` de validation de 0.8549. Après cette étape, le modèle final `MultinomialNB` est entraîné avec ces meilleurs paramètres.

Les performances sont évaluées avec *l'accuracy*, le classification report et la confusion matrix sur le jeu de test. Enfin, les prédictions et les matrices de confusion sont exportées dans des fichiers CSV et PNG pour faciliter l'analyse des résultats.

4.2. KNN (K-Nearest Neighbors)

L'implémentation complète de cet algorithme se trouve dans les fichiers [1 entraînement.py](#), [2 validation.py](#) et [3 test.py](#).

Les données de train, validation et test sont chargées avec *Pandas*, puis les textes sont prétraités avec la fonction *pretraitement* afin de nettoyer les critiques (suppression des balises HTML, normalisation du texte) avant l'apprentissage. Les critiques sont ensuite transformées en vecteurs numériques avec *TfidfVectorizer* (TF-IDF, 10 000 features maximum), et les matrices sparse sont sauvegardées au format *.npz* pour être réutilisées par les scripts suivants.

Plusieurs valeurs de K sont testées par une approche de recherche progressive, en partant d'une exploration large [1, 5, 10, 50, 100, 200, 400, 600, 800, 1000] puis en affinant autour des zones prometteuses, jusqu'à converger vers **K=345**. La sélection du meilleur K repose sur le **F1-score weighted** calculé sur le jeu de validation, afin d'assurer une cohérence méthodologique avec les critères d'évaluation finale. La distance **cosinus** est utilisée comme métrique de similarité, particulièrement adaptée aux vecteurs TF-IDF de haute dimension.

Les performances finales sont évaluées sur le jeu de test avec l'accuracy, le classification report et la confusion matrix. Le modèle obtient une **accuracy de 83.05%** et un **F1-score weighted de 0.83**, avec une légère asymétrie entre les deux classes : le modèle identifie mieux les critiques négatives (F1=0.84) que les critiques positives (F1=0.82). Enfin, la matrice de confusion est exportée en fichier PNG (*knn_matrice_confusion.png*) pour faciliter l'analyse des résultats.

4.3. WEKA / J48

Nous avons entraîné le modèle J48 sur le jeu d'entraînement et utilisé le jeu de validation comme jeu de développement pour ajuster les hyperparamètres.

Avec les paramètres par défaut, l'accuracy était de 73,96 % et la F-mesure pondérée de 0,740, ce que nous jugeons insuffisant. Nous avons donc procédé à un ajustement manuel de deux paramètres ayant le plus d'impact : *minNumObj* (de 2 à 20), qui limite la croissance de l'arbre et réduit le surapprentissage, et *confidenceFactor* (de 0,25 à 0,1), qui renforce l'élégage et simplifie le modèle.

Cette configuration améliore l'accuracy à 77 % et le Kappa à 0,54. Les métriques sont également symétriques entre les deux classes (precision, recall et F1 = 0,770), ce qui indique une absence de biais envers l'une ou l'autre classe. WEKA ne proposant pas de recherche automatique d'hyperparamètres et chaque entraînement étant coûteux en temps, nous retenons cette configuration comme paramétrage final.

Le modèle est ensuite sauvegardé avec la fonction *Save model* de WEKA, puis réutilisé pour les prédictions sur le jeu de test. Les résultats détaillés sont présentés dans la section 5.

4.4. WEKA / Random forest

Nous entraînons d'abord le modèle Random Forest sur le jeu de données de train avec WEKA. Dans les options de test, nous utilisons le jeu de validation afin d'évaluer les performances du modèle. Les résultats obtenus montrent que le recall, la précision et la f-mesure sont tous supérieurs à 0.8. Nous considérons donc que les paramètres par défaut offrent déjà des performances satisfaisantes.

Le modèle est ensuite sauvegardé avec la fonction *Save model* de WEKA, puis réutilisé pour effectuer les prédictions sur le jeu de test. Les résultats détaillés de l'évaluation sont présentés dans la section suivante.

5. Analyse des Résultats et Comparaison

Model	Class	Precision	Recall	F1	Accuracy
scikit-learn					
Naive Bayes	positive	0.85	0.882	0.87	86.45%
	negative	0.88	0.847	0.86	
	weighted avg	0.865	0.865	0.865	
KNN	positive	0.85	0.80	0.82	83.05%
	negative	0.81	0.86	0.84	
	weighted avg	0.83	0.83	0.83	
WEKA					
Random Forest	positive	0.834	0.861	0.847	84.49%
	negative	0.856	0.829	0.842	
	weighted avg	0.845	0.845	0.845	
J48 (C4.5) tuned	positive	0.775	0.777	0.776	77.60%
	negative	0.777	0.775	0.776	
	weighted avg	0.776	0.776	0.776	

Le tableau ci-dessus présente les performances des quatre modèles évalués sur le jeu de test. Globalement, tous les modèles atteignent une précision (*accuracy*) supérieure à 77 %, ce qui représente un seuil satisfaisant pour une tâche de classification binaire de sentiments. Il convient de noter que les modèles WEKA utilisent *StringToWordVector* pour la vectorisation, tandis que les modèles

scikit-learn utilisent **TF-IDF**, ce qui peut partiellement expliquer les écarts de performance observés.

Naive Bayes obtient la meilleure accuracy (86,45 %) et le meilleur F1 pondéré (0,865), malgré sa simplicité algorithmique. Ce résultat s'explique notamment par l'utilisation de bigrammes (`ngram_range = (1,2)`) et d'un alpha faible (0,1), qui permettent de mieux capturer les expressions multi-mots caractéristiques des critiques de films.

Random Forest affiche des performances proches (84,49 %, F1 = 0,845) avec un avantage notable sur la qualité des probabilités prédites : son ROC Area de 0,925 est le plus élevé de tous les modèles WEKA, ce qui indique une excellente capacité à distinguer les deux classes. C'est également le seul modèle WEKA dont les paramètres par défaut suffisent à obtenir de bonnes performances.

KNN obtient 83,05 % d'accuracy avec un F1 de 0,83. On observe une légère asymétrie entre les classes : le modèle identifie mieux les critiques négatives (F1 = 0,84) que positives (F1 = 0,82). Ce résultat a été obtenu après une recherche progressive du meilleur K, qui converge vers K = 345 avec une distance cosinus.

J48 présente les performances les plus faibles (77,60 %, F1 = 0,776), malgré un ajustement manuel des paramètres (`minNumObj = 20`, `confidenceFactor = 0,1`). Son ROC Area de 0,837 reste correct mais est nettement inférieur à celui de Random Forest (0,925), ce qui confirme que J48 est moins fiable dans l'estimation des probabilités de classe. En revanche, ses prédictions sont parfaitement équilibrées entre les deux classes, sans biais notable. Les limites de J48 s'expliquent par la nature du problème : un arbre de décision unique a du mal à capturer la complexité du langage naturel, contrairement aux méthodes ensemblistes comme Random Forest.

En conclusion, Naive Bayes se démarque comme le modèle le plus performant sur cette tâche, ce qui confirme son efficacité bien établie pour la classification de textes. Random Forest offre quant à lui le meilleur compromis entre performance et robustesse probabiliste.

6. Conclusion

Ce projet nous a permis de comparer plusieurs approches classiques de classification supervisée appliquées à l'analyse de sentiments sur le corpus IMDB.

Les résultats montrent que les méthodes probabilistes et les méthodes d'ensemble restent les plus efficaces sur ce type de tâche. Le modèle Naive Bayes obtient les meilleures performances avec une accuracy de 86,45 %, suivi par Random Forest (84,49 %). KNN atteint des résultats corrects (83,05 %), mais avec un coût de calcul plus important lors des prédictions. À l'inverse, J48 apparaît plus limité pour traiter des données textuelles de grande dimension, malgré l'ajustement manuel de plusieurs paramètres.

Cette expérimentation nous a également permis d'observer l'impact concret des outils utilisés. Les modèles développés avec scikit-learn et ceux entraînés avec WEKA n'utilisent pas exactement les mêmes méthodes de vectorisation (TF-IDF contre `StringToWordVector`), ce qui peut influencer les performances obtenues. La comparaison ne porte donc pas uniquement sur les algorithmes eux-mêmes, mais aussi sur les pipelines proposés par chaque environnement.

Nous avons également constaté certaines limites communes à l'ensemble des modèles testés. Les représentations utilisées restent principalement fondées sur une approche sac-de-mots : les modèles prennent en compte la fréquence des termes, mais comprennent difficilement le contexte, l'ironie ou certaines formulations ambiguës. De plus, les contraintes de temps de calcul nous ont conduits à limiter l'exploration des hyperparamètres sous WEKA. Par ailleurs, les méthodes linéaires telles que les SVM (Support Vector Machines) ou la régression logistique, considérées comme des références incontournables en classification de texte, n'ont pas été intégrées à nos expérimentations, ce qui restreint la portée comparative de nos résultats.

Malgré ces limites, les résultats obtenus restent satisfaisants pour une approche classique de classification de sentiments. Cette expérimentation nous a surtout permis de mieux comprendre les différences de comportement entre plusieurs algorithmes et outils appliqués au traitement automatique des textes.